

Legal RAG

Legal Assistant for Indian Case Law

Project Proposal

Dheeraj Murthy

Mathew Joseph

Ayush Tiwari

International Institute of Information Technology Bangalore

June 9, 2026

Abstract

India's legal ecosystem produces massive volumes of judgments that are difficult to search manually. Existing legal research tools are expensive and not optimized for Indian legal language. We present a Retrieval-Augmented Generation system that ingests 46,000+ Supreme Court judgment PDFs, indexes them as dense vector embeddings in PostgreSQL with pgvector, and answers legal queries using a two-stage retrieval pipeline followed by local LLM inference. The architecture combines BGE dense embeddings, cross-encoder reranking, and Qwen2.5-7B-Instruct to deliver grounded, citation-anchored answers without external API calls.

Project Snapshot

- Corpus: 46,000+ Supreme Court Judgments
- Vector Store: PostgreSQL + pgvector
- Embeddings: BGE Base v1.5
- Reranker: MiniLM Cross Encoder
- LLM: Qwen2.5-7B-Instruct
- Deployment: Docker Compose

Contents

1	Problem Statement	1
2	Background	1
3	Vision and Objectives	1
3.1	Vision	1
3.2	Core Objectives	1
3.3	Out of Scope	1
4	Datasets	2
5	System Architecture	2
5.1	Two-Stage Retrieval Pipeline	2
5.2	System Components	3
6	Tech Stack	4
7	Database Schema	4
8	Software Requirements	5
8.1	Functional Requirements	5
8.2	Non-Functional Requirements	5
9	Deployment	5
9.1	Docker Compose (Recommended)	5
9.2	Manual Setup	6
9.3	API Endpoints	6
9.4	CLI Usage	6
10	Evaluation Methodology	7
11	Project Status	7
12	Future Work	7

1 Problem Statement

India's legal ecosystem produces massive volumes of judgments, appeals, and legal commentary. However:

- Legal texts are long, complex, and difficult to search manually.
- Lawyers and students spend hours reading case law without tooling support.
- Existing legal research tools (Westlaw, SCC Online) are expensive and not optimized for Indian legal language and citation patterns.

There is a clear need for an open, accessible NLP system that understands Indian legal terminology, retrieves relevant precedents with high recall, and generates grounded answers with verifiable citations.

2 Background

This project began as a Project Elective under Prof. Tulika at IIIT Bangalore, focused on building NLP systems for the Indian legal scene. Over the previous semester, we developed a 2-stage RAG pipeline accumulating data from over 50,000 Indian Supreme Court judgments spanning 75+ years, using a Qwen Instruct model to create a chatbot that understands Indian law and answers queries relevant to the Indian Penal Code and jurisdiction.

As part of our research and market analysis, we conducted surveys and interviews with students, professors, and working alumni from reputed National Law Universities including NLSIU Bangalore and NUSRL Ranchi.

3 Vision and Objectives

3.1 Vision

Build a production-ready NLP system tailored to Indian legal language that enables fast semantic search, grounded question-answering, summarization, and insights over court judgments.

Target users: Lawyers, law students, legal researchers, LegalTech startups, and citizens checking case context.

3.2 Core Objectives

- Build a searchable case-law knowledge base from 46,000+ Supreme Court judgment PDFs.
- Deliver grounded Q/A over legal queries with verifiable case citations.
- Provide high-quality summaries of judgments with quoted passages.
- Facilitate petition generation from case context and legal principles.
- Create a secure, scalable web API and chat UI.

3.3 Out of Scope

- Predicting case outcomes.
- Replacing qualified legal professionals.

4 Datasets

The system uses three datasets:

1. LLM Fine Tuning Dataset of Indian Legal Texts Curated question-answer pairs from the Indian Penal Code (IPC), Criminal Procedure Code (CRPC), and the Indian Constitution. Used for instruction-tuning evaluation and prompt development.

<https://www.kaggle.com/datasets/akshatgupta7/llm-fine-tuning-dataset-of-indian-legal-texts>

2. Indian Supreme Court Judgments A CSV index of judgments extracted from the government API, including PDFs and metadata (court, case number, date, language). Updated weekly. Metadata includes a language column; some entries span multiple years.

<https://www.kaggle.com/datasets/vangap/indian-supreme-court-judgments>

3. Legal Dataset: SC Judgments India (1950–2024) 46,000+ PDFs covering approximately 98% of Supreme Court judgments available on Indian Kanoon. Primary corpus for the vector knowledge base, spanning 75+ years of Indian legal history. Each file represents an official judgment delivered by the Supreme Court of India.

<https://www.kaggle.com/datasets/adarshsingh0903/legal-dataset-sc-judgments-india-19502024>

5 System Architecture

5.1 Two-Stage Retrieval Pipeline

The system follows a four-step pipeline:

- Stage 1 – High-Recall Retrieval:** User query is embedded with BAAI/bge-base-en-v1.5 (768-dimensional dense vectors). PostgreSQL/pgvector performs cosine similarity search using an HNSW index. Returns 30 candidate chunks at a similarity threshold of 0.2.
- Stage 2 – Cross-Encoder Reranking:** A cross-encoder (cross-encoder/ms-marco-MiniLM-L-6-v2) scores each (query, chunk) pair. The top 8 chunks are selected for context.
- Generation:** Selected chunks are formatted into a Qwen chat-template prompt with strict grounding instructions. The LLM runs with `temperature=0.2`, `do_sample=False` (greedy decoding) for deterministic output grounded only in the provided context.
- Post-Processing:** Citations are extracted from response text, confidence scores are computed from embedding similarity and lexical overlap, and quality metrics are reported per query.

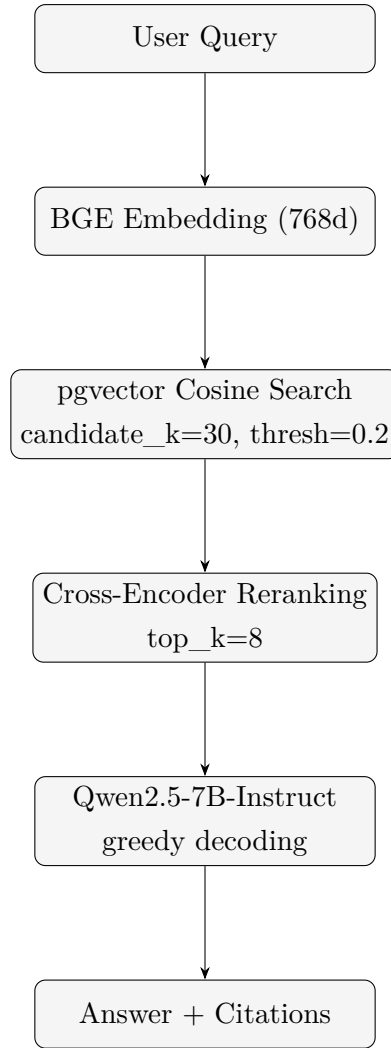


Figure 1: End-to-End RAG Pipeline

5.2 System Components

The FastAPI server orchestrates five modular components:

- **Retriever** (`retriever.py`): Connects to PostgreSQL, generates query embeddings via sentence-transformers, executes cosine similarity search with HNSW index.
- **Reranker** (`reranker.py`): Loads a CrossEncoder model, scores candidate chunks, returns top-N by relevance.
- **Prompt Builder** (`prompt_builder.py`): Formats retrieved chunks into a Qwen-instruct template with a strict system prompt: “Answer ONLY using the provided context.”
- **LLM Inference** (`llm_inference.py`): Loads Qwen2.5-7B-Instruct-1M via HuggingFace transformers with `device_map="auto"` and `torch.float16`.
- **Post-Processor** (`post_processor.py`): Cleans responses, extracts citations, computes confidence scores, and formats output.

6 Tech Stack

Layer	Technology
Backend Framework	Python 3.11, FastAPI, Uvicorn
Large Language Model	Qwen/Qwen2.5-7B-Instruct-1M
Embeddings Model	BAAI/bge-base-en-v1.5 (768d)
Cross-Encoder Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2
Vector Database	PostgreSQL 16 + pgvector (HNSW index)
PDF Ingestion	pdftotext (poppler-utils), PyPDF2
Optical Character Recognition	Tesseract + pytesseract + pdf2image
Frontend	Next.js (chatbot-ui) + bare HTML/JS fallback
Containerization	Docker Compose (postgres, api, frontend)
Hardware	GPU with ~16 GB VRAM (NVIDIA)

Table 1: Technology Stack

7 Database Schema

Three tables form the core schema. Judgments are split into sections (facts, issues, arguments, ratio, judgment) and each chunk is embedded as a 768-dimensional vector for cosine similarity search.

```

1 CREATE TABLE judgments (
2     id SERIAL PRIMARY KEY,
3     petitioner TEXT,
4     respondent TEXT,
5     court TEXT,
6     date_of_judgment DATE,
7     bench TEXT[],
8     citations JSONB,
9     judgment_text TEXT
10 );
11
12 CREATE TABLE judgment_chunks (
13     chunk_id SERIAL PRIMARY KEY,
14     judgment_id INTEGER REFERENCES judgments(id)
15     ON DELETE CASCADE,
16     section TEXT CHECK (section IN (
17         'facts', 'issues', 'arguments',
18         'ratio', 'judgment'
19     )),
20     content TEXT
21 );

```

```
22
23 CREATE TABLE judgment_embeddings (
24     chunk_id INTEGER PRIMARY KEY
25     REFERENCES judgment_chunks(chunk_id)
26     ON DELETE CASCADE,
27     embedding vector(768) NOT NULL
28 );
29
30 CREATE INDEX judgment_embedding_hnsw_idx
31 ON judgment_embeddings
32 USING hnsw (embedding vector_cosine_ops);
```

8 Software Requirements

8.1 Functional Requirements

- Answer legal questions with grounded case citations:
 - Search judgments by semantic similarity.
 - Refuse or warn when answer is unsupported by retrieved context.
 - Include “not legal advice” disclaimer.
- Show authoritative supporting cases: court, year, paragraph ID.
- Display quoted passages from judgments, not just references.
- Preserve traceability through chunk-to-judgment mapping.
- Handle conversation: follow-up questions via chat mode, clarification when no results found.

8.2 Non-Functional Requirements

- **Performance:** Retrieval in ~ 200 ms, generation in ~ 2 – 4 s. Scales with corpus size via HNSW index.
- **Accuracy:** Strict context-only prompting prevents hallucination. Confidence scoring combines embedding similarity and lexical overlap.
- **Security:** Self-hosted GPU inference. No external API calls for legal data. Zero user data logged externally.
- **Maintainability:** Modular pipeline design. Per-query metrics and debug information available.
- **UX:** Consistent response format across CLI, API, and web UI.

9 Deployment

9.1 Docker Compose (Recommended)

```
1 docker-compose -f deploy/docker/docker-compose.yml up -d
```

Service	Port	Description
postgres	5432	PostgreSQL 16 + pgvector
api	8000	FastAPI backend (Uvicorn)
frontend	3000	Next.js chat interface

9.2 Manual Setup

```

1 # 1. Initialize database
2 bash deploy/db_setup/init_db.sh
3
4 # 2. Ingest judgment PDFs
5 python database/ingest.py --input /path/to/pdfs
6
7 # 3. Start API server
8 cd rag_model && python api.py
9
10 # 4. Query via CLI
11 cd rag_model && python main.py \
12     --query "principles_of_natural_justice?"

```

9.3 API Endpoints

Endpoint	Method	Description
/query	POST	Single grounded query
/chat	POST	Multi-turn chat with memory
/chat/clear	POST	Clear conversation history
/chat/history	GET	Retrieve chat history
/document	POST	OCR + query on PDF/image
/retrieval-test	POST	Debug retrieval (no LLM)
/status	GET	System configuration

9.4 CLI Usage

```

1 # Single query
2 python main.py --query "What_are_the_principles_of_natural_justice?"
3
4 # Interactive mode
5 python main.py --interactive
6
7 # Chat with memory

```

```

8 python main.py --chat
9
10 # Retrieval-only (no GPU needed)
11 python main.py --retrieval-test

```

10 Evaluation Methodology

- **Recall@K**: Proportion of relevant chunks retrieved in the top-K candidates for each query.
- **Mean Reciprocal Rank (MRR)**: Average reciprocal rank of the first relevant result across test queries.
- **Citation Accuracy**: Fraction of generated citations that map to valid, retrieved chunks in the context.
- **Response Groundedness**: Lexical overlap between the generated answer and the provided context chunks.
- **End-to-End Latency**: Total time from query submission to response delivery, measured across the pipeline stages.

11 Project Status

Component	Status	Completion
Core RAG Pipeline	Complete	100%
Database Layer	Complete	100%
FastAPI Backend	Complete	100%
Next.js Frontend	Complete	100%
Docker Deployment	Complete	100%
OCR Support	Complete	100%
High Court Scraper	In Progress	60%
CI/CD Pipeline	Planned	0%
Production Monitoring	Planned	0%

Table 2: Project Status Overview

12 Future Work

- **Petition Generation**: Build templates and LLM-guided drafting workflows for legal petitions grounded in retrieved case law.
- **Hybrid Retrieval**: Combine dense vector search with sparse keyword retrieval (BM25) for improved recall.

- **Multilingual Support:** Extend to handle Indian regional language judgments (Hindi, Kannada, Tamil, etc.).
- **High Court Integration:** Expand beyond Supreme Court to include Karnataka, Delhi, Bombay, and other High Courts.
- **Citation Graph Search:** Enable traversal of citation networks between judgments.
- **Fine-tuned Legal Models:** Instruction-tune a smaller LLM specifically on Indian legal question-answer pairs.
- **Automated Evaluation:** Build a benchmark suite with curated legal queries and ground-truth citations.

Disclaimer

This system is a research tool and does not constitute legal advice.